SIGNALFORGE

Stage 1: Architecture Audit

Lead Finder Integration Readiness Report

Date: 2026-10-07
Scope: Read-only inspection of the existing SignalForge codebase
Purpose: Determine integration points, reusable infrastructure, conflicts, and gaps before implementing the Lead Finder module
Status: NO APPLICATION CODE WAS MODIFIED

1. Executive Summary

SignalForge is a mature, well-architected AI-powered outbound sales platform built as a **pnpm/Turborepo modular monorepo**. The codebase spans 58 database models, 200+ API endpoints, 15 BullMQ workers, and 81 frontend pages across 17 implemented modules.

The existing architecture is **highly favorable for Lead Finder integration**. Specifically:

- A **complete lead discovery pipeline** already exists (`packages/lead-discovery/`) with adapter-based source abstraction, 6-tier deduplication, normalization, email verification, ICP qualification, and a staging table (`DiscoveredLead`) that separates raw discoveries from canonical leads.
- The **Website Intelligence system** (`packages/website-scanner/`) is a standalone library with SSRF protection, 8 analyzers, AI enrichment, and graceful degradation — directly reusable.
- The **AI abstraction** (`packages/ai/`) provides a clean `AIProvider` interface with mock/openai providers and OpenRouter compatibility via `baseURL` override — FreeLLMAPI can plug in as a new adapter.
- The **queue infrastructure** (`packages/queue/` + BullMQ/Redis) is battle-tested with 20 named queues, configurable concurrency, exponential backoff, and idempotent job IDs.
- A **Lead Finder from Google Maps** requires only: (1) a new `GooglePlacesAdapter` implementing the existing `LeadSourceAdapter` interface, (2) FreeLLMAPI adapter for the AI layer, and (3) frontend pages following established patterns.

The primary gap is the **absence of a Google Maps/Places adapter** — the adapter interface exists and is ready, but no implementation ships. Secondary gaps include no runtime AI response validation, no AI usage tracking, and no global API rate limiting.

2. Current Tech Stack

```
Frontend:           Next.js 15.3 (App Router), React 19.2, TypeScript 5.8
UI Components:      shadcn/ui + Radix UI primitives, Tailwind CSS v4, Lucide icons
State Management:   TanStack React Query v5 (server), React useState/Context (client)
Forms:              react-hook-form v7, Zod validation (shared package)
Charts:             Recharts v3
Flows:              @xyflow/react (automation workflow editor)

Backend:            Fastify v5.3, TypeScript, ESM
Database:           PostgreSQL 16 via Prisma 7 (driver adapter pattern, @prisma/adapter-pg)
Queue:              BullMQ 5.53 + Redis 7 (ioredis 5.6)
AI:                 OpenAI SDK 4.86 (mock + openai providers, OpenRouter-compatible)
Email:              Custom SMTP provider (nodemailer-based)
File Parsing:       ExcelJS (XLSX), csv-parse (CSV), cheerio (HTML)
Website Scanning:   Custom crawler (native fetch, BFS, cheerio)

Auth:               Cookie-based sessions, RBAC (OWNER/ADMIN/MEMBER/VIEWER)
Tenancy:            Workspace-scoped (workspaceId on every business model)

Package Manager:    pnpm 9.12
Monorepo:           Turborepo
Node:               &gt;= 22.0.0
Deployment:         Docker Compose (PostgreSQL 16, Redis 7, Mailpit)
```

3. Repository Architecture

```
signalforge/
■■■ apps/
■   ■■■ api/                    # Fastify v5 REST API (200+ endpoints)
■   ■   ■■■ src/
■   ■       ■■■ app.ts          # App builder, plugin registration
■   ■       ■■■ server.ts       # Entry point
■   ■       ■■■ plugins/auth.ts # Cookie session auth, RBAC
■   ■       ■■■ routes/         # 20 route modules
■   ■       ■■■ services/       # Import processor, etc.
■   ■■■ web/                    # Next.js 15 frontend (81 pages)
■       ■■■ src/
■           ■■■ app/            # App Router (auth, onboarding, app groups)
■           ■■■ components/     # 111+ components (26 shadcn/ui primitives)
■           ■■■ lib/            # API client (3900+ lines), utilities
■■■ packages/
■   ■■■ ai/                     # AI abstraction (AIProvider interface, mock + openai)
■   ■■■ allocation/             # SMTP pool allocation and capacity reservation
■   ■■■ analytics/              # Analytics aggregation service
■   ■■■ auth/                   # Auth utilities, session management, RBAC
■   ■■■ automations/            # Workflow engine (event-driven, graph execution)
■   ■■■ campaigns/              # Campaign lifecycle, enrollment, sequencing
■   ■■■ config/                 # Zod-validated environment configuration
■   ■■■ database/               # Prisma schema (2942 lines, 58 models, 43 enums)
■   ■■■ deliverability/         # Email health, bounce tracking, domain checks
■   ■■■ email/                  # SMTP provider, email rendering
■   ■■■ inbound/                # Inbound email processing
■   ■■■ inbox/                  # Reply classification, conversation management
■   ■■■ integrations/           # Google Drive, file parsing, column detection
■   ■■■ lead-discovery/         # Discovery pipeline, adapters, normalization, dedup
■   ■■■ lead-research/          # LinkedIn research, provider chain, sanitization
■   ■■■ mailboxes/              # Mailbox lifecycle, health, warmup
■   ■■■ queue/                  # BullMQ factory, Redis connection, job definitions
■   ■■■ sending/                # Sending scheduler, job service, policy evaluation
■   ■■■ shared/                 # Shared utilities
■   ■■■ validation/             # Zod schemas (shared frontend + backend)
■   ■■■ verification/           # Email verification engine
■   ■■■ warmup/                 # Email warmup service
■   ■■■ website-scanner/        # Website crawler, 8 analyzers, scoring
■■■ workers/
■   ■■■ analytics-worker/
■   ■■■ deliverability-worker/
■   ■■■ email-worker/
■   ■■■ enrichment-worker/
■   ■■■ inbound-worker/
■   ■■■ inbox-reply-worker/
■   ■■■ lead-discovery-worker/
■   ■■■ lead-research-worker/
■   ■■■ scheduler-worker/       # Central tick-based scheduler (30s interval)
■   ■■■ verification-worker/
■   ■■■ warmup-worker/
■   ■■■ website-worker/
■   ■■■ workflow-worker/
■■■ services/
■   ■■■ scrapegraph/            # ScrapeGraph AI service (empty/planned)
■■■ infrastructure/
■   ■■■ deployment/
■   ■■■ docker/
■   ■■■ nginx/
```

```
■■■ docker-compose.yml          # PostgreSQL 16, Redis 7, Mailpit
■■■ package.json
■■■ pnpm-workspace.yaml
■■■ turbo.json
```

4. Existing Lead Architecture

4.1 Lead Model

The `Lead` model (schema line 390, 28 fields) is workspace-scoped with comprehensive contact, company, and geographic data:

Field	Type	Required	Purpose
id	String (cuid)	Yes	Primary key
workspaceId	String	Yes	Tenant isolation (FK → Workspace)
email	String	Yes	Contact email
firstName	String	No	First name
lastName	String	No	Last name
fullName	String	No	Display name
company	String	No	Company name
companyDomain	String	No	Company domain
jobTitle	String	No	Job title
industry	String	No	Industry classification
phone	String	No	Phone number
linkedinUrl	String	No	LinkedIn profile URL
countryCode	String	No	ISO country code
country	String	No	Country name
region	String	No	Region/area
state	String	No	State/province
city	String	No	City
website	String	No	Company website
icpScore	Int	No	ICP fit score
status	LeadStatus	Yes	NEW/CONTACTED/REPLIED/QUALIFIED/etc.
source	LeadSource	Yes	MANUAL/CSV_IMPORT/GOOGLE_DRIVE/DISCOVERY/API/OTHER
customFields	Json	Yes	Extensible fields (default "{}")
importId	String	No	Import tracking

Field	Type	Required	Purpose
sourceFileId	String	No	Source file tracking
createdAt	DateTime	Yes	auto
updatedAt	DateTime	Yes	auto

Unique constraint: `@@unique([workspaceId, email])` — one lead per email per workspace.

Indexes (11 total): workspaceId+status, workspaceId+source, workspaceId+companyDomain (annotated for dedup), workspaceId+countryCode, workspaceId+countryCode+state, workspaceId+countryCode+state+city, workspaceId+createdAt, workspaceId+importId, workspaceId+industry, workspaceId+jobTitle, workspaceId+status+createdAt.

Status: REUSABLE — The Lead model has all fields needed for Lead Finder output. The `LeadSource.DISCOVERY` enum value already exists. The `companyDomain` index is annotated in the schema specifically for domain-keyed deduplication scans.

4.2 Lead Relations (14 total)

Relation	Target	Type
workspace	Workspace	Many-to-one
verification	LeadVerification	One-to-one (optional)
enrichment	LeadEnrichment	One-to-one (optional)
campaignLeads	CampaignLead[]	One-to-many
tags	LeadTag[]	Many-to-many (junction)
listMembers	LeadListMember[]	Many-to-many (junction)
notes	LeadNote[]	One-to-many
websiteAnalyses	WebsiteAnalysis[]	One-to-many
scrapeGraphJobs	ScrapeGraphJob[]	One-to-many
sendingJobs	SendingJob[]	One-to-many
conversations	Conversation[]	One-to-many
outcomeEvents	LeadOutcomeEvent[]	One-to-many
workflowExecutions	WorkflowExecution[]	One-to-many
sourceRecords	LeadSourceRecord[]	One-to-many
discoveredFrom	DiscoveredLead[]	One-to-many
research	LeadResearch[]	One-to-many

Status: REUSABLE — All necessary relation types already exist. The `discoveredFrom` relation connects canonical leads back to their discovery source records.

4.3 Lead Creation Paths

Currently leads enter SignalForge through three paths:

Path	Entry Point	Staging	Dedup	Normalize	Verify	Qualify
Manual	`POST /leads/` API route	No	Prisma unique constraint	No	No	No
Smart Import	`POST /leads/smart-import`	No (direct to Lead)	6-tier DedupIndex	Yes	No	No
Discovery (CSV/XLSX/Mock)	Discovery pipeline → Promote	Yes (DiscoveredLead)	6-tier DedupIndex	Yes	Yes	Yes

The **Discovery path** is the most mature and is the correct integration point for Lead Finder. It uses a staging table (`DiscoveredLead`) and requires explicit promotion to canonical leads, which gives users control over what enters their CRM.

Status: REUSABLE — The discovery pipeline's adapter-based architecture means Lead Finder only needs a new adapter.

4.4 Lead Deduplication

Location: `packages/lead-discovery/src/dedup.ts`

Status: REUSABLE — Comprehensive 6-tier deduplication system.

The dedup system uses 6 tiers evaluated in strict priority order (design bias: under-merging preferred over wrongly merging):

Tier	Name	Logic
1	EMAIL	Exact match on normalized email
2	DOMAIN	Exact match on normalized domain (from website or email)
3	PHONE	Exact match on normalized phone
4	SOURCE_ID	Same source:sourceId pair (job re-run idempotency)
5	COMPANY_LOCATION	Identical normalized company name in same city/country
6	FUZZY_COMPANY	Levenshtein similarity >= 0.92, same city/country, name >= 6 chars

Tiers 5 and 6 refuse merge if both records have different domains.

`loadDedupCandidates()` fetches existing workspace leads via indexed OR query on batch emails/domains/phones/cities (capped at 5,000 rows). `dedupeBatch()` deduplicates against both existing workspace leads AND within the incoming batch.

This is already used by both the discovery pipeline and the smart-import processor.

---

5. Existing Queue Architecture

5.1 Technology

```
Queue Technology:       BullMQ 5.53
Worker Technology:      BullMQ Worker (Node.js)
Storage:                Redis 7 Alpine
Redis Connection:       ioredis 5.6, singleton per URL, maxRetriesPerRequest: null
Default Concurrency:    5
Default Retries:        3 attempts, exponential backoff, 1s base
Job Persistence:        Redis (removeOnComplete: 100-500, removeOnFail: 500-1000)
Dead-letter Handling:   No explicit DLQ; failures persisted to DB + retained in Redis
Scheduling:             Scheduler-worker (setInterval 30s tick, not BullMQ repeat/cron)
Monitoring:             Structured JSON logging; no BullMQ dashboard
```

5.2 Queue Map (20 named queues)

Queue	Worker	Concurrency	Retry	Backoff Base
`email-send`	email-worker	5	3	60s
`email-inbound`	inbound-worker	5	3	60s
`inbox-reply`	inbox-reply-worker	5	3	120s
`warmup`	warmup-worker	5	3	300s
`lead-discovery`	lead-discovery-worker	5	3	30s
`lead-enrichment`	enrichment-worker	5	3	1s
`lead-research`	lead-research-worker	1*	2	30s
`email-verification`	verification-worker	10*	3	5s
`website-analysis`	website-worker	5	3	1s
`deliverability-aggregation`	deliverability-worker	5	3	60s
`domain-health-check`	deliverability-worker	5	2	120s
`analytics`	analytics-worker	5	3	60s
`workflow`	workflow-worker	5	3	30s
`scheduler-tick`	scheduler-worker (producer)	N/A	N/A	N/A
`health`	email-worker (heartbeat)	N/A	N/A	N/A
`lead-processing`	No worker (unused)	-	-	-
`ai`	No worker (unused)	-	-	-
`notifications`	No worker (unused)	-	-	-
`lead-import`	No worker (unused)	-	-	-
`email-retry`	No worker (unused)	-	-	-

*Configurable via environment variables.

5.3 Lead Finder Queue Reusability

Requirement	Current Support	Assessment
Lead discovery jobs	`lead-discovery` queue exists	REUSABLE

Requirement	Current Support	Assessment
Website analysis	`website-analysis` queue exists	REUSABLE
AI enrichment	`lead-enrichment` queue exists	REUSABLE
Email verification	`email-verification` queue exists	REUSABLE
Concurrency control	Default 5, configurable per worker	REUSABLE
Retry with backoff	Exponential backoff on all queues	REUSABLE
Idempotency	Deterministic job IDs on all queues	REUSABLE
Persistence	Redis + DB dual persistence	REUSABLE
Progress reporting	DB status transitions (not BullMQ progress)	REUSABLE WITH EXTENSION
Cancellation	DB status check in processor (not BullMQ)	REUSABLE
Rate limiting	Application-level, not BullMQ limiter	REUSABLE WITH EXTENSION

Status: REUSABLE — The queue infrastructure can support Lead Finder without modification. The `lead-discovery` queue and worker already handle the exact flow needed.

---

6. Existing Website Intelligence

6.1 Architecture

```
Input:              URL (from Lead's website or companyDomain)
                         ↓
Crawler:            Native fetch, BFS, 10 pages max, 10s timeout per page
                         ↓
Analyzers (8):      technical, performance, SEO, mobile, content, conversion,
                    technology (40+ signatures), AI signals, modernity
                         ↓
Scoring:            Weighted composite (tech 20%, SEO 20%, perf 15%,
                    mobile 15%, content 15%, conversion 15%)
                         ↓
Opportunity:        8 rule-based detectors → top 3 opportunities
                         ↓
AI Enrichment:      aiProvider.analyzeWebsite() — summary, key issues,
                    opportunities, personalization hooks
                         ↓
Database:           WebsiteAnalysis model (30+ fields, JSON blobs for each category)
```

6.2 Input/Output

Input: URL (normalized), associated with a Lead ID. The `/analyze/url` endpoint auto-creates leads from bare URLs.

Output: WebsiteAnalysis record containing:

- `websiteScore` (0-100), `opportunityScore` (0-100), `opportunityPriority` (LOW/MEDIUM/HIGH/VERY_HIGH)
- Score breakdown per category
- Full findings for each analyzer

- Technologies detected (40+ signatures)
- AI-generated summary, key issues, opportunities, personalization hooks
- Recommended services with evidence

6.3 Protections

- **SSRF:** Multi-layer protection — scheme restriction, credential stripping, hostname blocklist, private IP range blocking, DNS resolution re-check (defends against DNS rebinding)
- **robots.txt:** Respected (User-agent: * directives)
- **Timeouts:** 10s per request, 5MB response limit, 5 max redirects
- **Rate limiting:** 1s inter-request delay within a scan; no cross-scan domain rate limit
- **Caching:** 24-hour cache per domain per workspace

6.4 Lead Finder Reuse Assessment

> ***Can Lead Finder send discovered websites into the existing Website Intelligence pipeline?***

YES. The promotion flow in `packages/lead-discovery/src/promotion.ts` already calls `enqueueWebsiteAnalysis()` for promoted leads with websites. The Website Intelligence pipeline accepts any Lead with a website URL.

The existing flow is:

```
DiscoveredLead (with website)
        ↓
promoteDiscoveredLeads()
        ↓
Creates canonical Lead
        ↓
enqueueWebsiteAnalysis(leadId)
        ↓
website-analysis queue
        ↓
website-worker
        ↓
WebsiteAnalysis record linked to Lead
```

Status: REUSABLE — No changes needed for basic integration. For pre-promotion website scoring (filtering before save), the `analyzeWebsite()` function in `packages/website-scanner/` could be called directly by the discovery pipeline without requiring a Lead record.

---

7. Existing AI Architecture

7.1 Abstraction

Interface: `AIProvider` in `packages/ai/src/types.ts` — 10 methods covering lead scoring, company research, email personalization, reply classification, deliverability analysis, analytics analysis, website analysis, and lead enrichment.

Providers:

- `MockAIProvider` — Deterministic mock for development (363 lines)

- `OpenAIProvider` — Uses official OpenAI SDK, `response_format: { type: 'json_object' }`, temperature 0.3, supports `baseURL` override (OpenRouter-compatible)

No other providers (Gemini, Groq, Mistral, Anthropic) exist.

7.2 Current AI Call Map

Module	AI Method	Provider	Purpose
Inbox (API)	`suggestReply`	mock/openai	Draft reply suggestions
Inbox (reply-classifier)	`classifyReply`	mock/openai	Classify ambiguous replies
Inbound Worker	`classifyReply`	mock/openai	Classify inbound replies
Deliverability (API)	`analyzeDeliverability`	mock/openai	Narrate deliverability metrics
Analytics (API)	`analyzeAnalytics`	mock/openai	Narrate outreach KPIs
Enrichment Worker	`enrichLead`	mock/openai	Full lead intelligence enrichment
Website Worker	`analyzeWebsite`	mock/openai	Website sales opportunity analysis

4 unused methods: `scoreLead`, `researchCompany`, `personalizeEmail`, `generateEmail` — implemented in both providers but never called.

7.3 Deterministic Scoring (Non-AI)

`packages/ai/src/scoring.ts` contains pure functions with no AI dependency:

- `inferSeniority(jobTitle)` — regex-based seniority classification
- `inferDepartment(jobTitle)` — regex-based department classification
- `calculateICPFitScore(lead, icp)` — weighted ICP fit scoring
- `calculateLeadQualityScore(lead, verification, hasWebsite)` — data completeness scoring
- `calculateProspectScore(icpFit, quality, opportunity, seniority)` — composite prospect score
- `buildDynamicVariables(lead, enrichment, website)` — email template variables

These are already used by both the discovery qualification pipeline and the enrichment worker.

7.4 Gaps

1. **No runtime AI response validation** — `JSON.parse(content) as T` cast without schema validation
2. **No retry/backoff** in the AI layer itself — relies on BullMQ job retries
3. **No usage/cost tracking** — token counts from API responses are discarded
4. **No streaming** — all calls are blocking await
5. **Provider creation not centralized** — `createAIProvider()` called independently in 7 locations
6. **Graceful degradation works well** — both enrichment and website workers catch AI failures and save deterministic results

7.5 FreeLLMAPI Integration Point

The target architecture:

```
Singleforge Module
       ↓
Singleforge AI Service (packages/ai/)
       ↓
AIProvider Interface (types.ts)
       ↓
FreeLLMAPIProvider (NEW adapter)
       ↓
FreeLLMAPI
       ↓
Configured Provider (Gemini, Groq, Mistral, etc.)
```

The existing `AIProvider` interface and `createAIProvider()` factory are directly reusable. FreeLLMAPI can be added as a new provider alongside `mock` and `openai` by:

1. Creating `packages/ai/src/freellmapi-provider.ts` implementing `AIProvider`
2. Adding `'freellmapi'` to the `AIProviderName` union type
3. Adding a case to the factory function in `packages/ai/src/factory.ts`
4. Adding `FREELLMAPI_BASE_URL` and `FREELLMAPI_API_KEY` env vars

Since the OpenAI provider already uses the OpenAI SDK with `baseURL` override, and FreeLLMAPI likely exposes an OpenAI-compatible API, it may be possible to reuse `OpenAIProvider` directly with `OPENAI_BASE_URL` pointed at FreeLLMAPI. This would require zero code changes.

Status: REUSABLE — The AI abstraction is a clean integration point.

---

8. Existing Excel/CSV Pipeline

8.1 Pipeline Flow

```
Upload (multipart or base64)
       ↓
MIME Resolution (CSV vs XLSX, Windows edge cases)
       ↓
Parse (csv-parse for CSV, ExcelJS for XLSX)
       ↓
Column Detection (~120 aliases → 19 lead fields + 6 custom)
       ↓
Value-inference for unmapped columns (email/phone/URL regex, 70% threshold)
       ↓
Row Mapping (column mappings → lead field values)
       ↓
Normalize (normalizeLead() from lead-discovery)
       ↓
Deduplicate (6-tier DedupIndex from lead-discovery)
       ↓
Create/Update Lead (fill-gaps merge for duplicates)
       ↓
Enqueue enrichment + research suggestions
```

8.2 Entry Points

Path	Route	Flow
Discovery Import	`POST /lead-discovery/jobs`	File → DiscoveredLead (staging) → manual promote

Path	Route	Flow
Smart Import	`POST /leads/smart-import`	File → Lead (direct, up to 10,000 rows)
Drive Import	`POST /drive/import`	Google Drive → background processing

8.3 Reusable Components

Component	Location	Status
`normalizeLead()`	`packages/lead-discovery/src/normalization.ts`	REUSABLE
`normalizeCompany/Email/Domain/Phone/Website/Country/State`	Same file	REUSABLE
`createDedupIndex()` / `findDuplicate()`	`packages/lead-discovery/src/dedup.ts`	REUSABLE
`loadDedupCandidates()`	`packages/lead-discovery/src/pipeline.ts`	REUSABLE
`qualifyLead()`	`packages/lead-discovery/src/qualification.ts`	REUSABLE
`parseFileStream()`	`packages/integrations/src/file-parser.ts`	REUSABLE
`detectLeadImportMappings()`	`packages/integrations/src/lead-import-field-mapper.ts`	REUSABLE
`mapRow()`	`packages/lead-discovery/src/adapters/file-adapter.ts`	REUSABLE
`promoteDiscoveredLeads()`	`packages/lead-discovery/src/promotion.ts`	REUSABLE
`enqueueWebsiteAnalysis()`	`packages/lead-discovery/src/enqueue-website-analysis.ts`	REUSABLE

Status: REUSABLE — The lead-discovery package already provides a unified ingestion pipeline that the Excel/CSV import also uses. Lead Finder should use the same pipeline.

---

9. Reusable Infrastructure

REUSE AS-IS

Service	Location	Why Reusable
Lead discovery pipeline	`packages/lead-discovery/src/pipeline.ts`	Complete flow: normalize → dedup → verify → qualify → persist
Lead normalization	`packages/lead-discovery/src/normalization.ts`	10+ normalizers covering all contact/company fields
6-tier deduplication	`packages/lead-discovery/src/dedup.ts`	Comprehensive, workspace-scoped, within-batch aware

Service	Location	Why Reusable
Lead qualification	`packages/lead-discovery/src/qualification.ts`	ICP scoring, seniority inference, prospect scoring
Lead promotion	`packages/lead-discovery/src/promotion.ts`	Staging → canonical lead with fill-gaps merge
Website Intelligence	`packages/website-scanner/`	Standalone library, 8 analyzers, SSRF-safe
AI abstraction	`packages/ai/`	Clean AIProvider interface, factory pattern
BullMQ infrastructure	`packages/queue/`	Factory functions, shutdown, job definitions
Email verification	`packages/verification/`	Full email verification with risk scoring
Lead enrichment worker	`workers/enrichment-worker/`	AI + deterministic scoring, input hashing
LeadSourceAdapter interface	`packages/lead-discovery/src/adapters/registry.ts`	Pluggable adapter pattern ready for Google Maps
DiscoveredLead staging table	`packages/database/prisma/schema.prisma`	Separate from Lead, full pipeline tracking fields
LeadSourceRecord provenance	Same file	Multi-source tracking per lead
LeadDiscoveryJob tracking	Same file	Full counters, status, configuration
LeadDiscoveryActivity log	Same file	Audit trail for discovery operations
Workflow event system	`packages/automations/`	Fire-and-forget event publishing with idempotency

REUSE WITH EXTENSION

Service	Location	Required Change	Reason
AI provider factory	`packages/ai/src/factory.ts`	Add FreeLLMAPI provider case	Only mock and openai exist
AIProviderName type	`packages/ai/src/types.ts`	Add `'freellmapi'` to union	Type safety for new provider
DiscoverySourceType enum	`schema.prisma`	Add `GOOGLE_PLACES` value	Currently has CSV/XLSX/MOCK/MANUAL + 3 planned
LeadSourceConfig model	`schema.prisma`	Configure Google Places per workspace	Already supports per-source enable/disable
Environment config	`packages/config/`	Add Google Maps API key, FreeLLMAPI vars	Zod-validated config schema
Frontend navigation	`app-sidebar.tsx`	Add Lead Finder nav item	Straightforward addition

DO NOT REUSE

Component	Location	Reason
ScrapeGraph service	`services/scrapegraph/`	Directory exists but is empty; source files not found in website-scanner/src

Component	Location	Reason
`scoreLead()` AI method	`packages/ai/`	Implemented but never called; behavior may not match Lead Finder needs
`researchCompany()` AI method	`packages/ai/`	Implemented but never called; would need validation

---

10. Missing Infrastructure

Component	Why Needed	Recommended Location
Google Maps/Places adapter	Core discovery source for Lead Finder	`packages/lead-discovery/src/adapters/google-places-adapter.ts`
FreeLLMAPI provider (or URL config)	AI gateway abstraction	`packages/ai/src/freellmapi-provider.ts` OR just configure `OPENAI_BASE_URL`
AI response validation	No runtime schema validation on AI responses	`packages/ai/src/openai-provider.ts` (add Zod parse)
AI usage tracking	Token counts discarded; no cost tracking	`packages/ai/src/usage-tracker.ts` + `AiUsageRecord` model exists
Global API rate limiting	Only auth routes are rate-limited	`apps/api/src/app.ts` (Fastify rate-limit on all routes)
Cross-scan domain rate limiting	Multiple scans can hit same domain simultaneously	`packages/website-scanner/` or worker-level
Lead Finder search UI	Search form with business type, location, depth	`apps/web/src/app/(app)/leads/finder/`
Lead Finder results UI	Results table with approve/reject/save actions	Same directory
Lead Finder search history UI	Past searches with status and results	Same directory

---

11. Architecture Conflicts

11.1 No Global Lead Creation Service

Lead creation happens in 3 separate locations (manual API route, smart-import processor, discovery promotion) with different normalization and dedup behavior. Lead Finder should use the discovery pipeline path exclusively.

Risk: LOW — The discovery pipeline is the most complete path and already handles Lead Finder's requirements.

11.2 AI Provider Instance Creation

`createAIProvider()` is called independently in 7 locations with the same env-var pattern, creating a new provider instance each time. Not a functional problem, but a missed centralization opportunity.

Risk: LOW — Each instance is lightweight (just an OpenAI SDK client).

11.3 Website Scanner Lead Coupling

The website-worker expects a `leadId` and writes to a Lead-associated `WebsiteAnalysis` record. If Lead Finder wants to score websites before creating leads (to filter out low-quality sites), it would need to call `analyzeWebsite()` from the scanner library directly, bypassing the worker.

Risk: MEDIUM — The scanner library function is callable directly, but the enrichment-worker integration (which triggers after website analysis) assumes a Lead exists.

11.4 File Payload Through Redis

The discovery worker carries base64-encoded file uploads in the BullMQ job payload (via Redis). For Google Maps results (potentially large JSON payloads), this pattern may need adjustment if results exceed Redis memory limits.

Risk: LOW — Google Maps API results are typically small (JSON, not binary files). The current pattern should work for up to 1,000 results.

---

12. Recommended Lead Finder Architecture

```
Lead Finder UI
       ↓
POST /lead-discovery/jobs
  (source: GOOGLE_PLACES, config: { query, location, radius, maxResults })
       ↓
lead-discovery queue
       ↓
lead-discovery-worker
       ↓
GooglePlacesAdapter.search()
       ↓
Google Maps Places API
       ↓
Raw Results (business name, address, phone, website, rating, category)
       ↓
normalizeLead() [EXISTING]
       ↓
dedupeBatch() [EXISTING]
       ↓
Email Discovery (website scraping for contact emails)
       ↓
verifyEmailFull() [EXISTING]
       ↓
qualifyLead() [EXISTING]
       ↓
DiscoveredLead staging table [EXISTING]
       ↓
User reviews results in UI
       ↓
POST /lead-discovery/jobs/:id/save
       ↓
promoteDiscoveredLeads() [EXISTING]
```

```
       ↓
Canonical Lead created
       ↓
enqueueWebsiteAnalysis() [EXISTING]    enqueueEnrichment() [EXISTING]
       ↓                                       ↓
website-analysis queue                  lead-enrichment queue
       ↓                                       ↓
Website Intelligence [EXISTING]         AI Qualification [EXISTING]
       ↓                                       ↓
AI via FreeLLMAPI [NEW ADAPTER]         AI via FreeLLMAPI [NEW ADAPTER]
       ↓                                       ↓
WebsiteAnalysis record                  LeadEnrichment record
       ↓
POST /lead-discovery/jobs/:id/enroll
       ↓
Campaign enrollment [EXISTING]
```

This architecture reuses ~90% of existing infrastructure. The only new components are:

1. `GooglePlacesAdapter` (implementing the existing `LeadSourceAdapter` interface)
2. FreeLLMAPI configuration (either as a new provider or via `OPENAI_BASE_URL`)
3. Frontend search/results UI

---

13. Future Data Flow

13.1 Search Creation

```
Input:  User submits { query: "dentists", location: "Austin, TX", radius: 25mi, maxResults: 100 }
Output: LeadDiscoveryJob record (status: PENDING)
Service: POST /lead-discovery/jobs → prisma.leadDiscoveryJob.create()
Queue:   lead-discovery queue → discovery-run job
```

13.2 Google Maps Discovery

```
Input:  LeadDiscoveryJob with source: GOOGLE_PLACES
Output: RawDiscoveryLead[] (business name, address, phone, website, rating, category, placeId)
Service: GooglePlacesAdapter.search() → Google Maps Places API (Text Search + Place Details)
Failure: Job marked FAILED, activity logged, user notified
```

13.3 Normalization

```
Input:  RawDiscoveryLead[]
Output: NormalizedLead[] (cleaned company, phone, website, domain, country, state, city)
Service: normalizeLead() from packages/lead-discovery/src/normalization.ts
Failure: Individual leads with invalid data skipped, counted in job.failed
```

13.4 Deduplication

```
Input:  NormalizedLead[] + existing workspace leads (up to 5,000)
Output: UniqueLead[] + duplicate count
Service: loadDedupCandidates() → createDedupIndex() → findDuplicate() per lead
Database: Lead table (indexed query on emails, domains, phones, cities)
Failure: N/A (pure function, no side effects)
```

13.5 Email Verification

```
Input:  UniqueLead[] with emails
Output: VerifiedLead[] with verification status, risk score
Service: verifyEmailFull() bounded by MAX_EMAIL_VERIFY_CONCURRENCY (default 5)
Queue:   email-verification queue for async verification
Failure: Verification failure logged, lead continues without verification
```

13.6 Qualification

```
Input:  VerifiedLead[] + workspace ICPProfile
Output: QualifiedLead[] with icpFitScore, qualityScore, prospectScore, seniority
Service: qualifyLead() using calculateICPFitScore(), calculateLeadQualityScore(), calculateProspectScore()
Database: ICPProfile lookup
Failure: Falls back to neutral score (50) if no ICP configured
```

13.7 Staging

```
Input:  QualifiedLead[]
Output: DiscoveredLead records (stage: QUALIFIED or READY)
Service: prisma.discoveredLead.upsert() keyed by jobId+source+sourceId
Database: DiscoveredLead table
Failure: Unique constraint violations handled gracefully
```

13.8 Promotion (User-Initiated)

```
Input:  User selects leads and clicks "Save to Workspace"
Output: Canonical Lead records + LeadSourceRecord provenance
Service: promoteDiscoveredLeads() → fill-gaps merge for existing leads
Queue:   website-analysis + lead-enrichment queued for each promoted lead
Database: Lead, LeadSourceRecord, WebsiteAnalysis, LeadEnrichment
Failure: Individual promotion failures logged, others continue
```

---

14. Database Changes Required

14.1 Schema Changes

MODIFY: DiscoverySourceType enum

```
// ADD:
GOOGLE_PLACES
```

The existing `DiscoveredLead` model already has a `source` field of type `DiscoverySourceType` and a `sourceId` field (String) — Google Maps `place_id` maps naturally to `sourceId`.

MODIFY: LeadDiscoveryJob model

No structural changes needed. The existing fields cover Lead Finder requirements:

- `source` → `GOOGLE_PLACES`
- `searchConfig` (Json) → `{ query, location, radius, maxResults }`
- `requirements` (Json) → `{ requireWebsite, requirePhone, requireEmail }`
- `totalDiscovered`, `totalUnique`, `totalDuplicates`, `totalQualified`, `totalFailed` → All counter fields exist

NEW: AiUsageRecord — NOT NEEDED (already exists)

The `AiUsageRecord` model already exists in the schema (line 1843) with fields for provider, model, operation, inputTokens, outputTokens, cost, latencyMs, and workspaceId. It is currently not populated by the AI layer but is ready for use.

Assessment

Proposed Entity	Existing Equivalent	New Model Needed?
LeadFinderSearch	LeadDiscoveryJob	NO — reuse existing
LeadFinderJob	Same as above	NO — reuse existing
LeadFinderResult	DiscoveredLead	NO — reuse existing
LeadEnrichment	LeadEnrichment	NO — already exists
LeadResearch	LeadResearch	NO — already exists
AIUsage	AiUsageRecord	NO — already exists

Only change: Add `GOOGLE_PLACES` to `DiscoverySourceType` enum.

---

15. API Changes Required

Existing Endpoints (no changes needed)

The `/lead-discovery/` route group already provides all necessary endpoints:

Endpoint	Purpose	Lead Finder Use
`POST /lead-discovery/jobs`	Create discovery job	Submit Google Maps search
`GET /lead-discovery/jobs`	List discovery jobs	Search history
`GET /lead-discovery/jobs/:id`	Get job detail	Search status
`GET /lead-discovery/jobs/:id/results`	Get discovered leads	Browse results
`POST /lead-discovery/jobs/:id/cancel`	Cancel job	Cancel search
`POST /lead-discovery/jobs/:id/retry`	Retry job	Retry failed search
`GET /lead-discovery/jobs/:id/activity`	Activity log	Search activity
`GET /lead-discovery/jobs/:id/export`	Export results	Export to CSV/XLSX
`POST /lead-discovery/jobs/:id/save`	Promote to workspace	Save selected leads
`POST /lead-discovery/jobs/:id/enroll`	Save + campaign enroll	Save and add to campaign
`GET /lead-discovery/sources`	List sources	Show available sources
`PUT /lead-discovery/sources`	Configure sources	Enable/configure Google Maps

New Endpoints: NONE REQUIRED

The existing API fully covers Lead Finder's needs. The `source` field in the job creation payload distinguishes Google Maps searches from file imports.

---

16. UI Changes Required

16.1 New Pages

Page	Path	Purpose
Lead Finder Search	`/leads/finder`	Search form: business type, location, radius, max results
Lead Finder Results	`/leads/finder/[jobId]`	Results table with scores, approve/reject, save actions
Lead Finder History	`/leads/finder/history`	Past searches with status badges

Alternative: Extend the existing `/leads/discovery` page with a "Google Maps" tab alongside the existing "File Import" tab. This avoids creating new navigation entries.

16.2 Reusable Components

Component	Location	Reuse For
`PageHeader`	`components/`	Page headers with actions
`StatCard`	`components/`	Result statistics
`EmptyState`	`components/`	No results state
`Table` + pagination	`components/ui/`	Results table
`Badge`	`components/ui/`	Status badges, score badges
`Progress`	`components/ui/`	Job progress indicator
`Dialog`	`components/ui/`	Confirmation dialogs
`Select` / `Input`	`components/ui/`	Search form fields
`VerificationStatusBadge`	`components/leads/`	Email verification status
`apiFetch()`	`lib/api.ts`	API communication
React Query patterns	Throughout	Data fetching, polling

16.3 Navigation

Add to sidebar Leads section in `components/layout/app-sidebar.tsx`:

```
{ title: 'Lead Finder', url: '/leads/finder', icon: Search }
```

Or extend the existing "Discovery" item to encompass both file import and Google Maps search.

---

17. Queue/Worker Changes Required

No New Queues or Workers Needed

The existing `lead-discovery` queue and `lead-discovery-worker` already handle the complete flow. The worker calls `runDiscoveryJob()` which dispatches to the appropriate adapter based on the job's `source` field.

Adding Google Maps support requires only:

1. Registering the new `GooglePlacesAdapter` in the adapter registry
2. The worker automatically picks it up via `getAdapter(source)`

Optional Enhancements

Enhancement	Purpose	Priority
BullMQ progress reporting	Show `42/100` in UI during Google Maps fetch	LOW — DB status transitions work
Domain-level rate limiting	Prevent concurrent website scans of same domain	MEDIUM — for large batches
Batch size limiting	Cap Google Maps results per job	LOW — API-level validation handles this

---

18. AI / FreeLLMAPI Integration Plan

Option A: URL Override (Zero Code Changes)

If FreeLLMAPI exposes an OpenAI-compatible API:

```
AI_PROVIDER=openai
OPENAI_BASE_URL=https://your-freellmapi-instance/v1
OPENAI_API_KEY=your-freellmapi-key
OPENAI_MODEL=your-preferred-model
```

This works because `OpenAIProvider` already passes `baseURL` to the OpenAI SDK constructor.

Option B: Dedicated Provider (More Control)

```
CREATE packages/ai/src/freellmapi-provider.ts
  - Implements AIProvider interface
  - FreeLLMAPI-specific error handling
  - Token usage tracking via AiUsageRecord
  - Model selection per operation (e.g., fast model for classification, capable model for enrichment)

MODIFY packages/ai/src/types.ts
  - Add 'freellmapi' to AIProviderName union

MODIFY packages/ai/src/factory.ts
  - Add freellmapi case to createAIProvider()

MODIFY packages/config/src/index.ts
  - Add FREELLMAPI_BASE_URL, FREELLMAPI_API_KEY, FREELLMAPI_MODEL env vars
```

```
MODIFY .env.example
  - Add FreeLLMAPI configuration section
```

Recommendation

Start with **Option A** (zero code changes) for immediate integration. Move to **Option B** when you need:

- Per-operation model selection
- Token usage tracking
- FreeLLMAPI-specific error handling
- Provider-specific retry logic

---

19. Security Considerations

19.1 API Keys

Key	Current Handling	Lead Finder Impact
OpenAI/FreeLLMAPI key	Server-side env var, never exposed to client	No change needed
Google Maps API key	NEW — must be server-side env var only	Add to `packages/config/` Zod schema
Session secret	Server-side env var	No change
Encryption key	Server-side env var (mailbox credentials)	No change

19.2 SSRF Protection

The Website Intelligence crawler has robust SSRF protection. Lead Finder inherits this for all website analysis. Google Maps API calls go to Google's endpoints (no user-controlled URLs), so no additional SSRF risk.

19.3 Tenant Isolation

All discovery jobs are scoped by `workspaceId` derived from the authenticated session (never from request params). The existing pattern ensures users cannot access other workspaces' searches, jobs, or discovered leads.

19.4 Job Authorization

Discovery jobs are created and accessed via workspace-authenticated endpoints. Job IDs are CUIDs (unguessable). The worker verifies workspace ownership before processing.

19.5 Lead Data

Google Maps data is publicly available business information. No PII concerns beyond standard lead data handling. The normalization pipeline already handles data cleaning.

19.6 Scraper Exposure

The Google Maps adapter calls the official Places API (not a scraper). No public endpoint is exposed. The API key is server-side only.

---

20. Performance Considerations

20.1 Scale Estimates

Scale	Google Maps API	Normalization	Dedup	Email Verify	Website Analysis	AI Enrichment
10 leads	1-2 API calls	<1s	<1s	~10s (5 concurrent)	~2min (sequential)	~30s (5 concurrent)
100 leads	5-10 API calls	<1s	<1s	~100s (5 concurrent)	~20min (5 concurrent)	~5min (5 concurrent)
500 leads	25-50 API calls	<2s	<2s	~500s (5 concurrent)	~100min (5 concurrent)	~25min (5 concurrent)
1,000 leads	50-100 API calls	<5s	<5s	~1,000s (5 concurrent)	~200min (5 concurrent)	~50min (5 concurrent)

20.2 Bottlenecks

1. **Website analysis** is the slowest stage (10s per page × 10 pages × sequential = ~100s per lead). At 1,000 leads with 5 concurrent workers, this takes ~5.5 hours.
2. **Email verification** at 5 concurrent is conservative. Could increase `MAX_EMAIL_VERIFY_CONCURRENCY` for batch operations.
3. **AI enrichment** depends on API latency. Mock provider is instant. OpenAI/FreeLLMAPI is ~3-5s per call.

20.3 Existing Infrastructure Capacity

Resource	Current Capacity	Lead Finder Impact
DB indexes	11 indexes on Lead, optimized for workspace-scoped queries	Adequate
Queue concurrency	5 per worker (configurable)	Adequate for 100-500; increase for 1,000+
Redis	Single instance, default config	Adequate; monitor memory for large job payloads
Website crawler	1s inter-request delay, 10 pages max	Conservative; adequate for concurrent scans
Dedup candidate loading	Capped at 5,000 rows	Adequate for most workspaces

---

21. Risks

Risk	Severity	Probability	Current Mitigation	Recommended Mitigation
Google Maps API key exposure	HIGH	LOW	N/A (key doesn't exist yet)	Server-side only; add to config Zod schema
Duplicate lead creation from concurrent jobs	MEDIUM	MEDIUM	6-tier dedup + Prisma unique constraint	Adequate — unique constraint is the backstop
AI provider failure blocks enrichment	MEDIUM	MEDIUM	Workers save deterministic results on AI failure	Adequate — graceful degradation works
Google Maps API rate limiting	MEDIUM	HIGH	None	Add exponential backoff in adapter; respect API quotas
Website crawler overload on target sites	MEDIUM	LOW	1s inter-request delay per scan	Add cross-scan domain rate limiting
Large job payloads in Redis	LOW	LOW	Base64 file in job payload	Google Maps JSON is small; monitor Redis memory
No AI response validation	MEDIUM	MEDIUM	TypeScript `as T` cast	Add Zod schema validation in AI provider
Missing global API rate limit	MEDIUM	LOW	Only auth routes rate-limited	Add Fastify rate-limit on all routes
Dedup candidate cap (5,000)	LOW	LOW	Hard cap in loadDedupCandidates	Increase cap or add pagination for large workspaces
No graceful shutdown on website/enrichment workers	LOW	LOW	Workers crash-restart cleanly	Add registerWorkerShutdown() calls
Long-running discovery jobs	MEDIUM	MEDIUM	No timeout on pipeline execution	Add job-level timeout; implement progress checkpointing

22. Architecture Scorecard

Area	Score / 10	Status	Explanation
Overall architecture	9	REUSABLE	Clean modular monorepo with clear package boundaries, Turborepo orchestration, consistent patterns

Area	Score / 10	Status	Explanation
Lead model	9	REUSABLE	Complete fields, proper indexes (including domain dedup index), workspace-scoped, extensible via customFields JSON
Lead ingestion	9	REUSABLE	Adapter-based discovery pipeline with staging table, fill-gaps promotion, workflow events
Deduplication	9	REUSABLE	6-tier system covering email /domain/phone/sourceId/co mpany+location/fuzzy, under-merge bias
Queue system	8	REUSABLE	BullMQ with sensible defaults, idempotent job IDs, exponential backoff; lacks BullMQ progress and DLQ
Website Intelligence	8	REUSABLE	Standalone library, 8 analyzers, SSRF protection, AI enrichment, 24h cache; lacks cross-scan rate limiting
AI abstraction	7	REUSABLE WITH EXTENSION	Clean interface, mock+openai providers, graceful degradation; no runtime validation, no usage tracking, only 2 providers
Excel pipeline	9	REUSABLE	Shared normalization/dedup with discovery, 120+ column aliases, value inference, fill-gaps merge
Multi-tenancy	9	REUSABLE	workspaceId on every model, derived from session (never client input), consistent enforcement across all routes
Error handling	7	PARTIALLY IMPLEMENTED	Workers handle errors well (graceful degradation); API lacks global error handler, inconsistent patterns across routes
Observability	5	PARTIALLY IMPLEMENTED	Structured JSON logging in workers; no AI usage tracking, no BullMQ dashboard, no metrics collection
Extensibility	9	REUSABLE	Adapter pattern for sources, provider pattern for AI, plugin-style route registration, workflow event system

Overall: 8.2 / 10 — The codebase is well-architected for Lead Finder integration.

---

23. Stage 2 Implementation Plan

Phase 1: AI Infrastructure (Optional — depends on FreeLLMAPI approach)

Objective: Connect FreeLLMAPI as an AI provider.

If using URL override (Option A):

- Zero code changes. Configure `OPENAI_BASE_URL` to point at FreeLLMAPI.

If using dedicated provider (Option B):

```
Files to create:
  packages/ai/src/freellmapi-provider.ts

Files to modify:
  packages/ai/src/types.ts         — Add 'freellmapi' to AIProviderName
  packages/ai/src/factory.ts       — Add factory case
  packages/config/src/index.ts     — Add env vars
  .env.example                     — Document FreeLLMAPI vars

Database changes: None
API changes: None
Queue changes: None

Dependencies: FreeLLMAPI instance running and accessible
Tests: Unit tests for provider, integration test with mock server
Acceptance: AI calls route through FreeLLMAPI successfully
```

Phase 2: Google Maps Adapter

Objective: Implement the Google Maps/Places data source.

```
Files to create:
  packages/lead-discovery/src/adapters/google-places-adapter.ts
  packages/lead-discovery/src/adapters/google-places-adapter.test.ts

Files to modify:
  packages/lead-discovery/src/adapters/registry.ts    — Register adapter
  packages/config/src/index.ts                        — Add GOOGLE_MAPS_API_KEY
  .env.example                                        — Document Google Maps vars

Database changes:
  schema.prisma: Add GOOGLE_PLACES to DiscoverySourceType enum
  Migration: Single enum addition

API changes: None (existing /lead-discovery/jobs endpoint handles all sources)
Queue changes: None (existing lead-discovery queue handles all sources)

Dependencies: Google Maps Places API access (API key)
Tests: Unit tests with mocked API responses; adapter interface compliance tests
Acceptance: GooglePlacesAdapter.search({ query, location }) returns normalized leads
```

Phase 3: Pipeline Integration

Objective: Ensure Google Maps results flow through the existing discovery pipeline.

```
Files to modify:
  packages/lead-discovery/src/pipeline.ts    — May need email discovery step
                                               for leads from Google Maps
                                               (Maps returns phone/website but
                                               rarely email)

Files to create:
  packages/lead-discovery/src/email-discovery.ts  — Extract emails from websites
                                                    (if not already handled)

Database changes: None
API changes: None
Queue changes: None

Dependencies: Phase 2 complete
Tests: Integration test: Google Maps results → normalize → dedup → verify → qualify → stage
Acceptance: Full pipeline processes 100 Google Maps results without errors
```

Phase 4: Email Discovery (If Needed)

Objective: Extract contact emails from business websites found via Google Maps.

Google Maps returns business info (name, address, phone, website, category) but typically not email addresses. We need an email discovery step that:

1. Fetches the business website (reusing website-scanner's fetcher with SSRF protection)
2. Extracts email addresses from page content (mailto links, text patterns)
3. Falls back to common patterns (info@domain, contact@domain)

```
Files to create:
  packages/lead-discovery/src/email-discovery.ts
  packages/lead-discovery/src/email-discovery.test.ts

Files to modify:
  packages/lead-discovery/src/pipeline.ts    — Add email discovery stage

Database changes: None (DiscoveredLead.emailSource already has EMAIL_DISCOVERED, MAILTO, PAGE_TEXT, PATTER
API changes: None
Queue changes: None

Dependencies: Phase 2 complete
Tests: Unit tests for email extraction from HTML, pattern generation
Acceptance: 30%+ of Google Maps leads have discovered emails
```

Phase 5: Lead Finder UI

Objective: Build the frontend for Lead Finder search and results.

```
Files to create:
  apps/web/src/app/(app)/leads/finder/page.tsx           — Search page
  apps/web/src/app/(app)/leads/finder/[jobId]/page.tsx   — Results page
  apps/web/src/app/(app)/leads/finder/history/page.tsx   — History page
  apps/web/src/components/lead-finder/search-form.tsx    — Search form
  apps/web/src/components/lead-finder/results-table.tsx  — Results table

Files to modify:
  apps/web/src/components/layout/app-sidebar.tsx  — Add Lead Finder nav item
  apps/web/src/lib/api.ts                         — Add Lead Finder API functions
                                                    (likely reuse existing
                                                    lead-discovery functions)
```

```
Database changes: None
API changes: None
Frontend changes: 3 new pages, 2 new components, sidebar update
Queue changes: None

Dependencies: Phase 2 + 3 complete (backend ready)
Tests: Component tests for search form, results table
Acceptance: User can search for businesses, view results, save to workspace
```

Phase 6: Campaign Integration

Objective: Allow users to enroll Lead Finder results directly into campaigns.

```
Files to modify: None — POST /lead-discovery/jobs/:id/enroll already exists

Dependencies: Phase 5 complete
Tests: E2E test: search → discover → save → enroll in campaign
Acceptance: Leads flow from Google Maps discovery into active campaigns
```

Phase 7: Testing & Hardening

Objective: Comprehensive testing and production readiness.

```
Tests:
  - Google Maps adapter: unit tests with mocked API
  - Pipeline integration: end-to-end with mock adapter
  - Dedup: test with Google Maps-style data (no emails, phone+domain dedup)
  - Email discovery: extraction accuracy tests
  - UI: component tests, search flow tests
  - Load: 100, 500, 1000 lead discovery jobs

Hardening:
  - Google Maps API error handling (quota, invalid key, rate limits)
  - Job timeout for long-running discoveries
  - Progress reporting to UI (poll-based via existing job status)
  - Cancellation support (existing cancel endpoint works)
```

---

24. Exact File-Level Changes

CREATE

```
packages/lead-discovery/src/adapters/google-places-adapter.ts
  Reason: Implements LeadSourceAdapter for Google Maps Places API.
          Handles search queries, pagination, place detail fetching,
          and mapping to RawDiscoveryLead format.

packages/lead-discovery/src/adapters/google-places-adapter.test.ts
  Reason: Unit tests with mocked Google Maps API responses.

packages/lead-discovery/src/email-discovery.ts
  Reason: Extract contact emails from business websites.
          Uses existing website-scanner fetcher (SSRF-safe).
          Checks mailto links, page text patterns, common email patterns.

packages/lead-discovery/src/email-discovery.test.ts
  Reason: Unit tests for email extraction logic.

apps/web/src/app/(app)/leads/finder/page.tsx
```

```
  Reason: Lead Finder search page with business type, location, depth inputs.

apps/web/src/app/(app)/leads/finder/[jobId]/page.tsx
  Reason: Lead Finder results page with discovered leads table, scores, save actions.

apps/web/src/app/(app)/leads/finder/history/page.tsx
  Reason: Lead Finder search history with status badges and result counts.

apps/web/src/components/lead-finder/search-form.tsx
  Reason: Search form component with Google Maps-specific fields.

apps/web/src/components/lead-finder/results-table.tsx
  Reason: Results table with lead scores, verification status, save/reject actions.
```

If FreeLLMAPI needs a dedicated provider (Option B):

```
packages/ai/src/freellmapi-provider.ts
  Reason: Implements AIProvider interface for FreeLLMAPI gateway.
```

MODIFY

```
packages/lead-discovery/src/adapters/registry.ts
  Reason: Register GooglePlacesAdapter in the adapter registry.

packages/database/prisma/schema.prisma
  Reason: Add GOOGLE_PLACES to DiscoverySourceType enum.

packages/config/src/index.ts
  Reason: Add GOOGLE_MAPS_API_KEY env var with Zod validation.

.env.example
  Reason: Document Google Maps API key configuration.

apps/web/src/components/layout/app-sidebar.tsx
  Reason: Add Lead Finder navigation item in the Leads section.

apps/web/src/lib/api.ts
  Reason: Add Lead Finder-specific API functions (or alias existing
          lead-discovery functions with Lead Finder-friendly names).

packages/lead-discovery/src/pipeline.ts
  Reason: Add email discovery stage for Google Maps source
          (between normalization and email verification).
```

If FreeLLMAPI needs a dedicated provider (Option B):

```
packages/ai/src/types.ts
  Reason: Add 'freellmapi' to AIProviderName union.

packages/ai/src/factory.ts
  Reason: Add freellmapi case to createAIProvider().
```

DO NOT TOUCH

```
packages/lead-discovery/src/dedup.ts              — Deduplication works as-is
packages/lead-discovery/src/normalization.ts       — Normalization works as-is
packages/lead-discovery/src/qualification.ts       — Qualification works as-is
packages/lead-discovery/src/promotion.ts           — Promotion works as-is
packages/website-scanner/                          — Entire package reusable as-is
packages/ai/src/openai-provider.ts                 — Works as-is (or via baseURL for FreeLLMAPI)
packages/ai/src/mock-provider.ts                   — Works as-is
packages/ai/src/scoring.ts                         — Works as-is
packages/queue/                                    — Queue infrastructure works as-is
```

```
workers/lead-discovery-worker/                    — Worker dispatches to adapter automatically
workers/website-worker/                           — Website analysis pipeline unchanged
workers/enrichment-worker/                        — Enrichment pipeline unchanged
workers/verification-worker/                      — Verification pipeline unchanged
apps/api/src/routes/lead-discovery.ts             — All needed endpoints exist
apps/api/src/plugins/auth.ts                      — Auth unchanged
packages/database/prisma/schema.prisma (models)   — No model changes needed
```

---

25. Acceptance Criteria

Before Stage 2 begins, verify:

- ■ This audit report has been reviewed and approved
- ■ Google Maps Places API access has been obtained (API key)
- ■ FreeLLMAPI approach decided (Option A: URL override vs Option B: dedicated provider)
- ■ FreeLLMAPI instance is running and accessible (if applicable)
- ■ Development environment is set up (PostgreSQL, Redis, Node 22)
- ■ All existing tests pass (`pnpm test`)
- ■ No application code was modified during Stage 1

Stage 2 completion criteria:

- ■ Google Maps search returns normalized business results
- ■ Results pass through dedup, email discovery, verification, and qualification
- ■ Users can view, filter, and save discovered leads from the UI
- ■ Saved leads appear in the leads list with source: DISCOVERY
- ■ Website Intelligence runs automatically on saved leads
- ■ AI enrichment runs automatically on saved leads
- ■ Leads can be enrolled in campaigns directly from Lead Finder results
- ■ All existing functionality continues to work (regression-free)

---

26. Final Recommendation

READY WITH CHANGES

The existing SignalForge architecture is **highly ready** for Lead Finder integration. The modular monorepo design, adapter-based discovery pipeline, comprehensive lead normalization/deduplication, and workspace-scoped tenancy model provide an excellent foundation.

Why READY:

- • The `LeadSourceAdapter` interface is designed exactly for this use case
- • The complete discovery pipeline (normalize → dedup → verify → qualify → stage → promote) exists and works
- • Website Intelligence, AI enrichment, and email verification are standalone and reusable
- • The queue/worker infrastructure handles the exact workload pattern needed

- Frontend patterns (data tables, filters, status badges, progress indicators) are well-established
- API endpoints for the full discovery lifecycle already exist

Required changes (minimal):

1. Implement `GooglePlacesAdapter` (~200-400 lines)
2. Add `GOOGLE_PLACES` to `DiscoverySourceType` enum (1 line + migration)
3. Register adapter in registry (1 line)
4. Add email discovery for Google Maps leads (~100-200 lines)
5. Build 3 frontend pages + 2 components (~500-800 lines)
6. Configure FreeLLMAPI (0 lines if using URL override)

Estimated implementation effort: 2-3 days for a developer familiar with the codebase.

No rewrites needed. No parallel infrastructure needed. No architectural changes needed.

Report generated: 2026-10-07

Status: Stage 1 complete. No application code was modified.

Verification: Read-only audit confirmed.

End of Stage 1 Architecture Audit